1. Problem Title & Source

Title: **Mirror Cipher**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

In a special encryption technique called **Mirror Cipher**, every letter in a message is replaced with its **mirror character in the English alphabet**.

The alphabet is mirrored as follows:

a ↔ z

b ↔ y

c ↔ x

d ↔ w

...

m ↔ n

You are given a **lowercase string S**.

Your task is to **convert each character in the string to its mirror character** and print the resulting encrypted string.

3. Examples

Example 1

Input

abc

Output

zyx

Explanation

a → z

b → y

c → x

Result

zyx

Example 2

Input

hello

Output

svool

Explanation

Original	Mirror
h	s
e	v
l	o
l	o
o	l

Encrypted message

svool

4. Constraints

1 ≤ length(S) ≤ 10^5

S contains only lowercase letters

Observations

- Alphabet size is fixed
- Mapping can be calculated mathematically

5. Core Concept (Pattern)

This problem involves:

Character transformation

ASCII arithmetic

Key insight:

mirror_char = 'a' + ('z' - current_char)

or

mirror_char = 'z' - (current_char - 'a')

6. Thought Process

Step 1 — Understand Alphabet Positions

Alphabet positions

a → 0

b → 1

...

z → 25

Mirror position

25 - position

Step 2 — Compute Mirror Character

Example

h

Position

7

Mirror

25 - 7 = 18

Character

s

7. Visualization

Alphabet mirror

a b c d e f g h i j k l m

z y x w v u t s r q p o n

Example

hello

Transformation

h → s

e → v

l → o

l → o

o → l

Output

svool

8. Pseudocode

```
read string S

for each character c in S

    mirror = 'z' - (c - 'a')

    append mirror to result

print result
```

9. Code Implementation

Python

```
s = input().strip()

result = ""

for c in s:
    mirror = chr(ord('z') - (ord(c) - ord('a')))
    result += mirror

print(result)
```

Java

```
import java.util.*;

class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        String s = sc.next();

        StringBuilder result = new StringBuilder();

        for(char c : s.toCharArray()){

            char mirror = (char)('z' - (c - 'a'));

            result.append(mirror);
        }

        System.out.println(result);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(N)

Where

N = length of string

Space Complexity

O(N)

For storing result.

11. Common Mistakes

1. Reversing string instead of mirroring characters
2. Incorrect ASCII arithmetic
3. Using wrong alphabet mapping
4. Handling uppercase incorrectly

12. Detailed Dry Run

Input

abc

Process

Character	Mirror
a	z
b	y
c	x

Result

zyx

13. Real-Life Applications

Mirror cipher concepts appear in:

- Cryptography
- Text encoding systems
- Substitution ciphers
- Puzzle encryption

14. Common Traps

1. Confusing with **string reverse**
2. Hardcoding alphabet mapping
3. Incorrect ASCII calculations
4. Missing edge cases

15. Related Problems

String manipulation problems:

LC 344 Reverse String

LC 917 Reverse Only Letters

LC 709 To Lower Case

16. Alternate Approaches

Approach	Complexity	Notes
ASCII calculation	O(N)	optimal
Prebuilt mapping table	O(N)	extra memory

17. Why This Solution Works

Because the English alphabet has **26 characters**, every character's mirror is simply **26 – position – 1**.

Using ASCII arithmetic allows us to compute this efficiently.

18. Variations / Follow-Up Questions

Possible interview variations:

1. Apply mirror cipher only to **vowels**
2. Encrypt **uppercase letters**
3. Combine **reverse + mirror**
4. Mirror only **every alternate character**